

Fast API and Uvicorn

What is FastAPI?

FastAPI is a modern, fast (high-performance) web framework for building APIs with Python 3.6+ based on standard Python type hints. It is designed for speed, ease of use, and automatic generation of interactive API documentation. FastAPI is ideal for building RESTful APIs, microservices, and backend systems.

- **High Performance:** Built on Starlette and Pydantic, FastAPI is one of the fastest Python frameworks available.
- **Type Safety:** Uses Python type hints for data validation and serialization.
- **Automatic Documentation:** Generates interactive API docs using Swagger UI and ReDoc.
- **Easy Integration:** Works seamlessly with modern Python libraries and tools.

Uvicorn

Uvicorn is a lightning-fast ASGI server implementation, using [uvloop](#) and [httptools](#). It is designed to run asynchronous web applications and is commonly used to serve FastAPI applications.

Installing FastAPI and Uvicorn

Prerequisites

Before you begin, ensure you have the following installed:

- Python 3.6 or later
- pip (Python package installer)

Step 1: Install FastAPI

You can install FastAPI using pip. Run the following command in your terminal:

```
pip install fastapi
```

Step 2: Install Uvicorn

Uvicorn is an ASGI web server implementation for Python.

Install Uvicorn using pip:

```
pip install uvicorn
```

Step 3: Verify Installation

To verify that FastAPI and Uvicorn are installed correctly, you can run the following commands:

```
pip show fastapi
pip show uvicorn
```

If the installation was successful, you should see information about the installed packages, including their versions.

Simple Endpoint

This section will cover the creation of a simple endpoint using FastAPI.

Step 1: Create a FastAPI Application

First, create a new Python file (e.g., `main.py`) and import FastAPI:

```
from fastapi import FastAPI

app = FastAPI()
```

Step 2: Define an Endpoint

Next, define a simple GET endpoint:

```
@app.get("/")
def read_root():
    return {"Hello": "World"}
```

Step 3: Run the Application

Finally, run the application using Uvicorn:

```
uvicorn main:app --reload
```

You can now access the endpoint at `http://127.0.0.1:8000/`.

Application will automatically launch at port 8000.

At default the application will be reloaded automatically when you make changes to the code.

Automatic API Documentation in FastAPI

FastAPI comes with **built-in, interactive documentation** powered by **Swagger UI** and **ReDoc**. This feature saves developers time and effort, as you don't need to manually create or maintain API documentation — FastAPI generates it automatically from your code.

Why Automatic Documentation Matters

- **No Extra Effort** – Just by defining your endpoints, FastAPI builds docs for you.
 - **Interactive Testing** – Swagger UI allows you to **execute requests directly from the browser**.
 - **Readable for All** – Developers, testers, and even non-technical stakeholders can explore endpoints easily.
 - **Always in Sync** – Docs update automatically when your code changes.
-

Documentation Options in FastAPI

FastAPI exposes **two interactive documentation UIs** out of the box:

1. Swagger UI

A friendly web interface that allows you to:

- Browse all endpoints
- View request/response models
- Try out endpoints with live API calls

Default URL:

```
http://127.0.0.1:8000/docs
```

2. ReDoc

A more structured and professional-looking API documentation interface, suitable for external users or clients. It provides:

- Collapsible endpoint sections
- Clear schema visualizations
- Enhanced readability for large APIs

Default URL:

```
http://127.0.0.1:8000/redoc
```

Customizing Documentation

FastAPI allows you to customize the generated documentation by using the description, title, and version parameters when creating the FastAPI app instance:

```
app = FastAPI(
    title="My API",
    description="This is a sample API",
    version="1.0.0"
)
```

Path Variables and Query Parameters

Path Variables

Path variables are used to capture values from the URL. In the example below, the `id` path variable is captured from the URL.

```
@app.get("/todos/{id}")
def read_item(id: int):
    return {"id": id}
```

Query Parameters

Query parameters are used to filter or modify the request. In the example below, the `skip` and `limit` query parameters are used to paginate the results.

```
@app.get("/todos/")
def read_items(skip: int = 0, limit: int = 10):
    return {"skip": skip, "limit": limit}
```

Demo

Add the following code to `main.py` to see the path and query parameters in action:

```
from fastapi import FastAPI

app = FastAPI()

# example for path parameters
# http://localhost:8000/todos/5
@app.get("/todos/{id}")
def read_item(id: int):
    return {"id": id}
```

```
# example for query parameters
# http://localhost:8000/todos/?skip=10&limit=10
@app.get("/todos/")
def read_items(skip: int = 0, limit: int = 10):
    return {"skip": skip, "limit": limit}
```

Run the application with the command:

```
uvicorn main:app --reload
```

Verify that the application is running by visiting the following URLs in your browser:

- For path parameters: <http://localhost:8000/todos/5>
- For query parameters: <http://localhost:8000/todos/?skip=10&limit=10>

Request Body Validation in FastAPI

When building APIs, it's crucial to ensure that incoming data is **valid, structured, and safe**. FastAPI integrates seamlessly with **Pydantic** to provide automatic **request body validation**, making your APIs more robust and developer-friendly.

What is Pydantic?

Pydantic is a Python library for **data parsing and validation** using Python type hints. FastAPI uses Pydantic models to:

- Define the expected structure of request and response data.
- Automatically validate and parse JSON payloads into strongly-typed Python objects.
- Provide helpful error messages if the input data is incorrect.

Think of Pydantic as a **contract** between your API and its clients — it ensures that data exchanged always matches the expected schema.

Example: Todo Model with Validation

```
from fastapi import FastAPI
from pydantic import BaseModel

# Define a request body model
class Todo(BaseModel):
    id: int
    title: str
    completed: bool
```

```

app = FastAPI()

# Example endpoint returning a Todo
@app.get("/todos/")
async def get_todos():
    todo = Todo(id=1, title="Learn FastAPI", completed=False)
    return {"todos": [todo]}

# Endpoint with request body validation
@app.post("/todos/")
async def create_todo(todo: Todo):
    return {"todo": todo}

```

Why Do We Need Request Body Validation?

1. Automatic Data Validation

- Ensures clients send data in the correct format.
- Example: If `id` must be an integer, FastAPI rejects `"abc"` automatically.

2. Clear Error Messages

- Invalid input triggers informative error responses.
- Example:

```

{
  "detail": [
    {
      "loc": ["body", "id"],
      "msg": "value is not a valid integer",
      "type": "type_error.integer"
    }
  ]
}

```

3. Type Safety & Developer Productivity

- Type hints + Pydantic models give IDEs **autocompletion** and **linting support**.
- Reduces runtime bugs.

4. Data Parsing Made Easy

- Incoming JSON automatically becomes Python objects.
- No need for manual parsing or validation.

Running the Demo

1. Save the code above in `main.py`.

2. Start the server with:

```
uvicorn main:app --reload
```

3. Test your API:

- Open **Swagger UI**: <http://127.0.0.1:8000/docs>
- Try **POST** `/todos/` with a valid payload:

```
{
  "id": 1,
  "title": "Learn FastAPI",
  "completed": false
}
```

- Try sending invalid data (e.g., `"id": "abc"`) and see the validation error.

Middleware in FastAPI

Middleware in FastAPI is a powerful mechanism to **intercept requests and responses** globally. It allows you to add cross-cutting functionality such as **logging, security, performance monitoring, or request transformations** without changing individual routes.

What is Middleware?

- Middleware runs **before** each request is processed by a path operation.
- Middleware runs **after** a response is generated, before sending it back to the client.

In short: **Every request → Middleware → Path operation → Middleware → Response.**

Common uses of middleware:

- Logging requests & responses
 - Authentication & authorization checks
 - Adding custom headers
 - Measuring performance
 - Enforcing HTTPS
-

Example 1: Simple Middleware with `@app.middleware`

```

from fastapi import FastAPI, Request
import time

app = FastAPI()

@app.middleware("http")
async def log_requests(request: Request, call_next):
    start_time = time.time()

    # Forward request to the next handler
    response = await call_next(request)

    process_time = time.time() - start_time
    formatted_process_time = f"{process_time:.2f}s"

    print(f"Request: {request.method} {request.url} completed in {formatted_process_time}")

    return response

@app.get("/")
async def read_root():
    return {"Hello": "World"}

```

Key points

- `request`: gives access to request details (headers, URL, body, etc.).
- `call_next(request)`: sends the request down to the actual route handler.
- Code before `call_next` runs **before the handler**.
- Code after `call_next` runs **after the handler, before response is sent**.

Example 2: Custom Middleware with `BaseHTTPMiddleware`

FastAPI is built on **Starlette**, which provides `BaseHTTPMiddleware` for creating reusable, class-based middleware.

```

import asyncio
from fastapi import FastAPI, Request
import time
from starlette.middleware.base import BaseHTTPMiddleware

app = FastAPI()

class LoggingMiddleware(BaseHTTPMiddleware):
    async def dispatch(self, request: Request, call_next):
        print("Request received at", time.strftime("%X"))
        response = await call_next(request)
        print("Response sent at", time.strftime("%X"))

```

```

        return response

# Register custom middleware
app.add_middleware(LoggingMiddleware)

@app.get("/")
async def read_root():
    await asyncio.sleep(3) # Simulate delay
    return {"Hello": "World"}

```

Key points

- Inherit from `BaseHTTPMiddleware`.
- Override the `dispatch` method to intercept request/response.
- Use `app.add_middleware()` to add it to the app.
- Useful when you need **stateful or reusable middleware classes**.

Example 3: Built-in Middleware (HTTPS & Trusted Hosts)

FastAPI also ships with **common built-in middleware** for security and best practices.

```

from fastapi import FastAPI
from fastapi.middleware.httpsredirect import HTTPSRedirectMiddleware
from fastapi.middleware.trustedhost import TrustedHostMiddleware

app = FastAPI()

# Redirect all HTTP → HTTPS
app.add_middleware(HTTPSRedirectMiddleware)

# Only allow requests from trusted hosts
app.add_middleware(TrustedHostMiddleware, allowed_hosts=["example.com",
                                                         "*.example.com"])

@app.get("/")
async def main():
    return {"message": "Hello Secure World"}

```

Key points

- `HTTPSRedirectMiddleware`: forces all traffic to HTTPS/WSS.
- `TrustedHostMiddleware`: blocks requests from untrusted domains.

Best Practices

- Use `@app.middleware` for **simple, inline middleware**.
- Use `BaseHTTPMiddleware` when building **reusable, complex middleware**.

- Avoid heavy computations inside middleware (they run on **every request**).
 - Keep middleware **stateless** whenever possible.
 - Use **built-in middlewares** (HTTPS, CORS, GZip, TrustedHost) instead of reinventing.
 - Log **essential details only** (method, path, status code, duration) to avoid noise.
-

Introduction to Swagger (OpenAPI) in FastAPI

FastAPI automatically generates **OpenAPI documentation** for your APIs, which powers the **Swagger UI** and **ReDoc** interactive docs.

This allows developers and testers to **explore, test, and understand APIs** without writing manual documentation.

What is Swagger (OpenAPI)?

- **OpenAPI** is a specification for describing APIs in a standard format (JSON/YAML).
- **Swagger UI** is a web-based interface that reads the OpenAPI schema and provides:
 - Interactive API documentation
 - Built-in request/response testing
 - Schema validation and descriptions

In FastAPI, your path operations, parameters, and models **automatically generate** the OpenAPI schema behind the scenes.

Example 1: Conditional OpenAPI (Enable/Disable Swagger)

Sometimes, you may want to disable Swagger docs in production for **security** or **performance** reasons. You can do this by controlling the `openapi_url` parameter:

```
from fastapi import FastAPI
from pydantic_settings import BaseSettings

# If openapi_url is empty, Swagger UI will not be available
class Settings(BaseSettings):
    openapi_url: str = "/openapi.json"

settings = Settings()

app = FastAPI(openapi_url=settings.openapi_url)

@app.get("/")
def root():
    return {"message": "Hello World"}
```

If `openapi_url` is `None` or `""`, no OpenAPI schema will be exposed, and Swagger docs are disabled.

Example 2: Extending the OpenAPI Schema

You can customize the generated OpenAPI schema by overriding FastAPI's `app.openapi` function:

```
from fastapi import FastAPI
from fastapi.openapi.utils import get_openapi

app = FastAPI()

@app.get("/todos/")
async def get_todos():
    return [{"name": "Learn FastAPI", "description": "A comprehensive guide to FastAPI"}]

def custom_openapi():
    if app.openapi_schema:
        return app.openapi_schema
    openapi_schema = get_openapi(
        title="Custom title",
        version="2.5.0",
        summary="This is a very custom OpenAPI schema",
        description="Here's a longer description of the custom **OpenAPI** schema",
        routes=app.routes,
    )
    # Add custom branding (logo in docs)
    openapi_schema["info"]["x-logo"] = {
        "url": "https://fastapi.tiangolo.com/img/logo-margin/logo-teal.png"
    }
    app.openapi_schema = openapi_schema
    return app.openapi_schema

app.openapi = custom_openapi
```

This allows you to:

- Change **title, version, description**.
- Add a **logo or vendor-specific metadata**.
- Extend schema with custom fields.

Example 3: Customizing Swagger UI

Swagger UI can also be customized using the `swagger_ui_parameters` parameter in `FastAPI()`.

```
from fastapi import FastAPI
```

```
# Disable syntax highlighting OR apply a theme
# app = FastAPI(swagger_ui_parameters={"syntaxHighlight": False})
app = FastAPI(swagger_ui_parameters={"syntaxHighlight": {"theme": "obsidian"}})

@app.get("/todos/{title}")
async def read_todo(title: str):
    return {"message": f"Hello {title}"}
```

Options you can customize:

- **syntaxHighlight** → Enable/disable or set a theme.
- **defaultModelsExpandDepth** → Expand/collapse models by default.
- **docExpansion** → Control how operations expand in the UI (**none**, **list**, **full**).
- **displayRequestDuration** → Show request duration in responses.

Best Practices (Trainer Notes)

- **Keep docs enabled in dev/test**, but consider disabling (**openapi_url=None**) in production.
- Use **custom OpenAPI schemas** to brand your API when sharing with external clients.
- Organize endpoints with **tags** for cleaner docs.
- Use **descriptions and summaries** in path operations for better readability in Swagger UI.
- Leverage **Pydantic models** for request/response schemas — they make docs self-explanatory.

Dotenv

In this example, we will use the **python-dotenv** package to load environment variables from a **.env** file. This is useful for managing sensitive information like API keys, database URLs, and other configuration settings without hardcoding them into your application.

Step 1: Install **python-dotenv**

You can install the **python-dotenv** package using pip. Run the following command in your terminal:

```
pip install python-dotenv
pip install pydantic-settings
```

pydantic-settings is used to manage settings and environment variables in a structured way.

Step 2: Create a **.env** File

Create a file named **.env** in the root directory of your project. Add your environment variables in the following format:

```
SECRET_KEY=supersecret
DEBUG=True
DATABASE_URL=mongodb://localhost:27017/mydatabase
API_KEY=your_api_key
```

Make sure to add `.env` to your `.gitignore` file to prevent it from being committed to version control.

Step 3: Load Environment Variables in Your FastAPI Application

In your FastAPI application, you can load the environment variables using the `load_dotenv` function from the `dotenv` module. Here's an example of how to do this:

```
from fastapi import FastAPI
from pydantic_settings import BaseSettings # pip install pydantic-settings
from dotenv import load_dotenv # pip install python-dotenv
import os

# Load environment variables from .env file
load_dotenv()
app = FastAPI()

class Settings(BaseSettings):
    SECRET_KEY: str = os.getenv("SECRET_KEY")
    DEBUG: bool = os.getenv("DEBUG", "False") == "True"
    DATABASE_URL: str = os.getenv("DATABASE_URL")
    API_KEY: str = os.getenv("API_KEY")

settings = Settings()
print(f"Loaded settings: {settings.json()}")

@app.get("/config")
def get_config():
    return {
        "SECRET_KEY": settings.SECRET_KEY,
        "DEBUG": settings.DEBUG,
        "DATABASE_URL": settings.DATABASE_URL,
        "API_KEY": settings.API_KEY,
    }

@app.get("/")
def read_root():
    return {"message": "Hello, World!"}

# To run the app, use the command: uvicorn main:app --reload
```

Step 4: Run Your FastAPI Application

You can run your FastAPI application using Uvicorn. Use the following command in your terminal:

```
uvicorn main:app --reload
```

Replace `main` with the name of your Python file (without the `.py` extension) if it's different.

Step 5: Test the Configuration Endpoint

You can test the `/config` endpoint to see if the environment variables are loaded correctly. Open your browser or use a tool like Postman or curl to access:

```
http://localhost:8000/config
```

You should see a JSON response with the values of your environment variables.

Conclusion

Using `python-dotenv` to manage environment variables in your FastAPI application helps keep sensitive information secure and makes it easier to configure your application for different environments (development, testing, production).

Make sure to handle your environment variables carefully and avoid exposing sensitive information in your codebase.

Project structure for a REST API

Structuring your FastAPI project is crucial for maintaining a clean and efficient codebase.

Benefits

Here are the most important reasons to structure your code based on best practices.

- **Enhanced Scalability:** Well-organized code grows smoothly with your FastAPI project.
- **Improved Maintainability:** Easy-to-understand code makes updates and fixes simple.
- **Streamlined Collaboration:** Clear structure helps teams work together efficiently.

Structuring Formats

FastAPI applications can be structured in different ways to accommodate various project needs.

There are two main approaches for structuring projects. One is based on file type and the other is based on module functionality.

1. Structuring based on File-Type

In this approach, files are organized by type (e.g., API, CRUD, models, schemas, routers) as represented by FastAPI itself.

This structure is more suitable for microservices or projects with fewer scopes:

```
.
├── app # Contains the main application files.
│   ├── __init__.py # this file makes "app" a "Python package"
│   ├── main.py # Initializes the FastAPI application.
│   ├── dependencies.py # Defines dependencies used by the routers
│   ├── routers
│   │   ├── __init__.py
│   │   ├── items.py # Defines routes and endpoints related to items.
│   │   └── users.py # Defines routes and endpoints related to users.
│   ├── services
│   │   ├── __init__.py
│   │   ├── item.py # Defines CRUD operations for items.
│   │   └── user.py # Defines CRUD operations for users.
│   ├── schemas
│   │   ├── __init__.py
│   │   ├── item.py # Defines schemas for items.
│   │   └── user.py # Defines schemas for users.
│   ├── models
│   │   ├── __init__.py
│   │   ├── item.py # Defines database models for items.
│   │   └── user.py # Defines database models for users.
│   ├── external_services
│   │   ├── __init__.py
│   │   ├── email.py # Defines functions for sending emails.
│   │   └── notification.py # Defines functions for sending notifications
│   └── utils
│       ├── __init__.py
│       ├── authentication.py # Defines functions for authentication.
│       └── validation.py # Defines functions for validation.
├── tests
│   ├── __init__.py
│   ├── test_main.py
│   ├── test_items.py # Tests for the items module.
│   └── test_users.py # Tests for the users module.
├── requirements.txt
├── .gitignore
└── README.md
```

2. Structuring based on Module-Functionality

In the second approach, we organize our files based on package functionality for example authentication sub-package, users sub-package, and posts sub-package.

The module-functionality structure is better suited for monolithic projects containing numerous domains and modules. By grouping all file types required by a single sub-package, it improves development efficiency.

In this structure, each package has its own router, schemas, models, etc.

```

fastapi-project
├── src
│   ├── auth
│   │   ├── router.py          # auth main router with all the endpoints
│   │   ├── schemas.py        # pydantic models
│   │   ├── models.py         # database models
│   │   ├── dependencies.py    # router dependencies
│   │   ├── config.py         # local configs
│   │   ├── constants.py      # module-specific constants
│   │   ├── exceptions.py     # module-specific errors
│   │   ├── service.py        # module-specific business logic
│   │   └── utils.py          # any other non-business logic functions
│   ├── aws
│   │   ├── client.py         # client model for external service communication
│   │   ├── schemas.py
│   │   ├── config.py
│   │   ├── constants.py
│   │   ├── exceptions.py
│   │   └── utils.py
│   ├── posts
│   │   ├── router.py
│   │   ├── schemas.py
│   │   ├── models.py
│   │   ├── dependencies.py
│   │   ├── constants.py
│   │   ├── exceptions.py
│   │   ├── service.py
│   │   └── utils.py
│   ├── config.py            # global configs
│   ├── models.py            # global database models
│   ├── exceptions.py        # global exceptions
│   ├── pagination.py        # global module e.g. pagination
│   ├── database.py          # db connection related stuff
│   └── main.py
├── tests/
│   ├── auth
│   ├── aws
│   └── posts
├── templates/
│   └── index.html
├── requirements
│   ├── base.txt
│   ├── dev.txt
│   └── prod.txt
├── .env
├── .gitignore
└── logging.ini

```

Conclusion

Structuring your FastAPI projects is essential for maintaining scalability, readability, and maintainability as your application grows. By organizing your code effectively, you ensure that it remains manageable and adaptable to evolving requirements.

TodoApp Modularization with FastAPI

- This project demonstrates how to **modularize a FastAPI application** by splitting the code into **models, schemas, routers, and services**.
 - This makes the project **clean, maintainable, and scalable** for real-world use.
-

Project Structure

```
TodoApp/
├── db/
│   └── db.json           # Database mock (JSON file)
├── models/
│   └── todoModel.py      # Python class representing a Todo object
├── routers/
│   └── todoRouter.py     # API routes for Todos
├── schemas/
│   └── todoSchema.py     # Pydantic schemas for validation & serialization
├── services/
│   └── todoService.py    # Business logic and database operations
├── main.py              # FastAPI application entry point
└── requirements.txt      # Dependencies
```

Database: db/db.json

A simple JSON file acting as our database:

```
[
  {
    "id": 1,
    "title": "Sample Todo",
    "completed": false
  },
  {
    "id": 2,
    "title": "Another Todo",
    "completed": true
  }
]
```

```
}  
]
```

Models: `models/todoModel.py`

The model defines the **Python object structure** for a Todo.

```
class Todo:  
    def __init__(self, id: int, title: str, completed: bool = False):  
        self.id = id  
        self.title = title  
        self.completed = completed
```

Schemas: `schemas/todoSchema.py`

Schemas are Pydantic models that handle **validation and serialization**.

```
from pydantic import BaseModel  
  
class TodoOut(BaseModel):  
    id: int  
    title: str  
    completed: bool  
  
    class Config:  
        orm_mode = True
```

Ensures data is valid before sending/receiving. Integrates with Swagger docs automatically.

Services: `services/todoService.py`

Business logic and database interaction (reading JSON file).

```
import json  
from pathlib import Path  
  
DB_PATH = Path(__file__).parent.parent / "db" / "db.json"  
  
def get_all_todos():  
    with open(DB_PATH, "r", encoding="utf-8") as f:  
        return json.load(f)
```

Routers: `routers/todoRouter.py`

Defines the **API endpoints** for Todos.

```
from fastapi import APIRouter
from schemas.todoSchema import TodoOut
from services import todoService as todo_service

router = APIRouter()

@router.get("/", response_model=list[TodoOut])
def list_todos():
    return todo_service.get_all_todos()
```

Uses `APIRouter` for modular route grouping. Applies response model validation with `TodoOut`.

Application Entry: `main.py`

The root FastAPI app where routers are included.

```
from fastapi import FastAPI
from routers import todoRouter

app = FastAPI(
    title="Todo App",
    version="1.0.0",
    description="Todo application API"
)

# Include todos router
app.include_router(todoRouter.router, prefix="/todos", tags=["todos"])
```

Requirements: `requirements.txt`

```
fastapi
uvicorn
pydantic
```

Running the Application

1. Install dependencies:

```
pip install -r requirements.txt
```

2. Run the server with **Uvicorn**:

```
uvicorn main:app --reload
```

3. Open in your browser:

- API Docs (Swagger): <http://127.0.0.1:8000/docs>
- ReDoc: <http://127.0.0.1:8000/redoc>
- Todos API: <http://127.0.0.1:8000/todos>

Best Practices

- Use **APIRouter** for modular endpoints (keeps routes organized).
- Separate **models** (internal representation) and **schemas** (validation & response).
- Keep **services** focused on business logic (reusable, testable).
- Use **db.json** only for demo — replace with **real database** (Postgres, MongoDB) in production.
- Add more routes (**POST**, **PUT**, **DELETE**) to complete CRUD functionality.

TodoApp CRUD with FastAPI

This project extends the modular TodoApp by implementing full CRUD operations:

- Create a new todo
- Read todos (list & single)
- Update an existing todo
- Delete a todo

New Schemas (**schemas/todoSchema.py**)

```
class TodoBase(BaseModel):
    title: str

class TodoCreate(TodoBase):
    pass

class TodoUpdate(BaseModel):
    title: str = None
    completed: bool = None
```

Explanation

- **TodoBase** → A shared base schema that defines fields common to multiple schemas (**title**).
 - **TodoCreate** → Used when creating a new Todo. Inherits from **TodoBase**.
 - **TodoUpdate** → Used for updating a Todo. Fields are optional (**title**, **completed**) to allow **partial updates**.
-

New Routes (**routers/todoRouter.py**)

```
@router.get("/{id}", response_model=TodoOut)
def get_todo(id: int):
    return todo_service.get_todo(id)

@router.post("/", response_model=TodoOut, status_code=201)
def create_todo(data: TodoCreate):
    return todo_service.create_todo(data)

@router.put("/{id}", response_model=TodoOut)
def update_todo(id: int, data: TodoUpdate):
    return todo_service.update_todo(id, data)

@router.delete("/{id}", status_code=204)
def delete_todo(id: int):
    return todo_service.delete_todo(id)
```

Explanation

- **GET /todos/{id}** → Fetch a single Todo by **id**.
 - **POST /todos/** → Create a new Todo. Uses **TodoCreate** schema for validation. Returns **201 Created**.
 - **PUT /todos/{id}** → Update an existing Todo. Uses **TodoUpdate** schema, allowing **partial updates**.
 - **DELETE /todos/{id}** → Deletes a Todo by **id**. Returns **204 No Content**.
-

Service Layer Updates (**services/todoService.py**)

Helper functions

```
def read_todos():
    if not DB_PATH.exists():
        return []
    with open(DB_PATH, "r", encoding="utf-8") as f:
        return json.load(f)

def write_todos(todos):
    with open(DB_PATH, "w", encoding="utf-8") as f:
        json.dump(todos, f, indent=2)
```

- `read_todos()` → Reads todos from the JSON file.
 - `write_todos()` → Saves todos back to the JSON file.
-

CRUD Operations

```
def get_todo(id: int):
    todos = read_todos()
    for todo in todos:
        if todo["id"] == id:
            return todo
    return None
```

- Finds a todo by ID. Returns `None` if not found.

```
def create_todo(data):
    todos = read_todos()
    new_id = max([todo["id"] for todo in todos], default=0) + 1
    todo = {"id": new_id, "title": data.title, "completed": False}
    todos.append(todo)
    write_todos(todos)
    return todo
```

- Creates a new todo with a **unique ID**.
- Defaults `completed = False`.

```
def update_todo(id: int, data):
    todos = read_todos()
    for todo in todos:
        if todo["id"] == id:
            if data.title is not None:
                todo["title"] = data.title
            if data.completed is not None:
                todo["completed"] = data.completed
            write_todos(todos)
            return todo
    return None
```

- Updates an existing todo.
- Only updates provided fields (supports **partial updates**).

```
def delete_todo(id: int):
    todos = read_todos()
    new_todos = [todo for todo in todos if todo["id"] != id]
```

```
write_todos(new_todos)
return None
```

- Removes a todo by ID.
- Returns nothing, but updates the database file.

Summary of CRUD Endpoints

Method	Endpoint	Description
GET	/todos/	List all todos
GET	/todos/{id}	Get todo by ID
POST	/todos/	Create new todo
PUT	/todos/{id}	Update existing todo
DELETE	/todos/{id}	Delete todo by ID

Custom Error Handling in TodoApp

In this update, we introduce **custom exception handling** to make error responses more consistent and user-friendly.

Custom Exception (`error/todoNotFound.py`)

```
class TodoNotFoundException(Exception):
    def __init__(self, todo_id: int):
        self.todo_id = todo_id
```

- Defines a custom exception `TodoNotFoundException`.
 - Accepts the missing `todo_id` so we can include it in the error message.
-

Exception Handler

```
def todo_not_found_exception_handler(request: Request, exc:
    TodoNotFoundException):
    return JSONResponse(
        status_code=status.HTTP_404_NOT_FOUND,
        content={"detail": f"Todo with id {exc.todo_id} not found."}
    )
```

- When the exception is raised, this handler returns:
 - **HTTP 404 Not Found** status.
 - JSON response like:

```
{ "detail": "Todo with id 5 not found." }
```

Raising the Exception in Routes (`routers/todoRouter.py`)

```
@router.get("/{id}", response_model=TodoOut)
def get_todo(id: int):
    todo = todo_service.get_todo(id)
    if not todo:
        raise TodoNotFoundException(id)
    return todo
```

- If a todo is missing, `TodoNotFoundException` is raised.
- The same pattern is applied in `update_todo` and `delete_todo` routes.

Registering the Handler (`main.py`)

```
app.add_exception_handler(TodoNotFoundException, todo_not_found_exception_handler)
```

- This registers our handler globally.
- Any route that raises `TodoNotFoundException` will automatically trigger the JSON error response.

Alternative: Middleware

```
@app.middleware("http")
async def todo_not_found_middleware(request: Request, call_next):
    try:
        response = await call_next(request)
        return response
    except TodoNotFoundException as exc:
        return todo_not_found_exception_handler(request, exc)
```

- Instead of using `add_exception_handler`, you could catch exceptions with middleware.
- Middleware wraps **every request/response cycle**.
- In this case, exception handlers are usually preferred because they are more explicit and modular.

Example Response

Request:

```
GET /todos/99
```

Response:

```
{
  "detail": "Todo with id 99 not found."
}
```

Testing TodoApp with Pytest

Testing ensures that our API and services **work as expected**, remain **reliable**, and **catch bugs early**. In this section, we integrate **pytest** into our TodoApp.

Why Testing is Important?

1. **Reliability** – Confirms your API and business logic behave correctly.
 2. **Maintainability** – Prevents accidental breakage when making changes.
 3. **Confidence** – Developers can refactor code without fear.
 4. **Automation** – Tests can run in CI/CD pipelines to ensure quality.
 5. **Documentation** – Tests serve as **living documentation** for how the system should behave.
-

What is Pytest?

- **pytest** is a powerful testing framework for Python.
 - Simple function-based tests, no boilerplate required.
 - Provides **fixtures** for reusable test setup.
 - Supports plugins (e.g., coverage, mocking).
 - Great integration with FastAPI's **TestClient** for API testing.
-

What is **assert**?

The **assert** keyword is used to verify that a condition holds true.

Example:

```
assert 2 + 2 == 4 # Passes
assert "hi".upper() == "HI" # Passes
```

```
assert 5 > 10 # Fails (raises AssertionError)
```

- If the condition is **True**, the test passes.
- If it's **False**, the test fails, showing the error.

What is **monkeypatch**?

- **monkeypatch** is a pytest fixture that lets you **temporarily replace** or **mock** functions, methods, or objects during a test.
- Useful when you don't want to perform real I/O (like file writes) or external API calls.

Example:

```
def get_data():
    return "real data"

def test_with_monkeypatch(monkeypatch):
    monkeypatch.setattr("__main__.get_data", lambda: "fake data")
    assert get_data() == "fake data"
```

Here, **get_data()** is overridden only for this test.

Test Files

1. API Tests (**tests/test_todo_api.py**)

Uses **FastAPI's TestClient** to test endpoints:

```
from fastapi.testclient import TestClient
from main import app

client = TestClient(app)

def test_list_todos():
    resp = client.get("/todos/")
    assert resp.status_code == 200
    assert isinstance(resp.json(), list)
```

Verifies API routes like **GET /todos/**, **POST /todos/**, **PUT /todos/{id}**, **DELETE /todos/{id}**. Checks both **success** and **failure** scenarios (e.g., todo not found).

2. Service Layer Tests (**tests/test_todo_service.py**)

Tests **business logic** without hitting the API:

```
@pytest.fixture
def sample_todos(monkeypatch):
    todos = [
        {"id": 1, "title": "Test 1", "completed": False},
        {"id": 2, "title": "Test 2", "completed": True}
    ]
    monkeypatch.setattr(todoService, "read_todos", lambda: todos.copy())
    monkeypatch.setattr(todoService, "write_todos", lambda x: None)
    return todos
```

Uses `monkeypatch` to mock file reads/writes. Verifies `get_all_todos`, `create_todo`, `update_todo`, and `delete_todo`. Ensures logic works independently of FastAPI.

Running Tests

Run all tests:

```
pytest -v
```

Run specific file:

```
pytest tests/test_todo_api.py -v
```

Run with detailed output:

```
pytest -s
```

Example Output

```
tests/test_todo_api.py::test_list_todos PASSED
tests/test_todo_api.py::test_get_todo_success PASSED
tests/test_todo_api.py::test_get_todo_not_found PASSED
tests/test_todo_service.py::test_create_todo PASSED
```

FastAPI Todo App - Authentication & Authorization

Overview

This project demonstrates how to implement **JWT-based authentication** and **role-based authorization** in a FastAPI application. It features a todo API with user management, secure login, and admin/user roles.

Authentication

- **JWT (JSON Web Token)** is used for stateless authentication.
- Users log in via `/auth/login` and receive a JWT token.
- The token must be included in the `Authorization: Bearer <token>` header for protected routes.

Signup

- New users can register via `/auth/signup`.
 - Default role for new users is `user`.
 - Admin user is initialized in `db/user.json`.
-

Authorization

- **Role-based access control** is enforced:
 - **Admin** (`role: admin`): Can create, update, delete todos and view/manage users.
 - **User** (`role: user`): Can only view todos (GET), cannot modify todos or access user management routes.

Route Protection

- All `/todos` routes require authentication.
 - Only admins can POST, PUT, DELETE todos.
 - Only admins can access `/users` routes.
 - Regular users can only GET todos.
-

Project Structure

```
TodoApp/
├── db/
│   ├── todo.json          # Stores todo records
│   └── user.json           # Stores user records
├── error/
│   ├── todoNotFound.py    # Custom error for Todo not found
│   └── userNotFound.py    # Custom error for User not found
├── models/
│   ├── todoModel.py       # Todo model class
│   └── userModel.py       # User model class
├── routers/
│   ├── authRouter.py      # Login & Signup endpoints
│   └── userRouter.py      # User management (admin-only)
```

```
├── todoRouter.py          # Todo CRUD (with role checks)
├── schemas/
│   ├── todoSchema.py     # Pydantic schemas for todos
│   └── userSchema.py      # Pydantic schemas for users
├── services/
│   ├── authService.py    # Authentication & JWT handling
│   └── todoService.py    # Business logic for todos
├── .env                  # Secret & config
├── main.py               # FastAPI entrypoint
└── requirements.txt      # Dependencies
```

Usage

1. Install dependencies:

```
pip install -r requirements.txt
```

2. Run the app:

```
uvicorn main:app --reload
```

3. Login:

- Use `/auth/login` with username and password to get a JWT token.
- Default admin credentials:
 - Username: `admin`
 - Password: `admin`

4. Signup:

- Use `/auth/signup` to create a new user.

5. Access protected routes:

- Include the JWT token in the `Authorization` header.

Example Requests

Login

```
POST /auth/login
Content-Type: application/x-www-form-urlencoded

username=admin&password=admin
```

Signup

```
POST /auth/signup
Content-Type: application/json

{
  "username": "newuser",
  "password": "newpass"
}
```

Access Todos (as user)

```
GET /todos/
Authorization: Bearer <your_token>
```

Create Todo (admin only)

```
POST /todos/
Authorization: Bearer <admin_token>
Content-Type: application/json

{
  "title": "New Task"
}
```

File Structure

- `main.py` - FastAPI app setup and router inclusion.
- `routers/` - API route definitions (`todoRouter.py`, `authRouter.py`, `userRouter.py`).
- `services/` - Business logic and helpers (`authService.py`, `todoService.py`).
- `db/` - JSON files for persistent storage (`user.json`, `todo.json`).
- `error/` - Custom exception handlers.

Security Notes

- Passwords are hashed using bcrypt via passlib.

- JWT secret key should be changed for production.
- Only admins can manage users and todos.

Monitoring & Logging in TodoApp

This module adds structured logging, request monitoring middleware, and health check endpoints to make the API production-ready.

Python Logging Setup (`utils/logging_config.py`)

We configure logging using Python's built-in `logging` module with both file-based and console handlers.

Key Features

- RotatingFileHandler:
Logs are written to `logs/app.log`. Old logs are rotated when the file reaches 1 MB, keeping 5 backups.
- Console Logging:
Logs are also streamed to the console during development.
- Log Format:

```
2025-09-08 05:21:47 INFO todoapp Application startup
2025-09-08 05:21:50 INFO todoapp [2c3f7b] -> 127.0.0.1 GET /todos
```

Example Code

```
import logging
from pathlib import Path
from logging.handlers import RotatingFileHandler

def setup_logging(name: str = "todoapp", level: int = logging.INFO, logs_dir: Path
| None = None) -> logging.Logger:
    # Place logs at project root: ../15_todoapp_monitor/logs/app.log
    root_dir = Path(__file__).resolve().parent.parent
    logs_dir = logs_dir or (root_dir / "logs")
    logs_dir.mkdir(exist_ok=True)

    logger = logging.getLogger(name)
    logger.setLevel(level)

    formatter = logging.Formatter("%(asctime)s %(levelname)s %(name)s %(message)s")

    file_handler = RotatingFileHandler(
```

```

    logs_dir / "app.log", maxBytes=1_000_000, backupCount=5, encoding="utf-8"
)
file_handler.setFormatter(formatter)

stream_handler = logging.StreamHandler()
stream_handler.setFormatter(formatter)

# Reset handlers to avoid duplicates on reload
logger.handlers.clear()
logger.addHandler(file_handler)
logger.addHandler(stream_handler)

# Reduce noise from uvicorn access logs
logging.getLogger("uvicorn.access").setLevel(logging.WARNING)
return logger

```

Request Logging Middleware (`main.py`)

Every request is wrapped with logging middleware that tracks:

- Request ID (`X-Request-ID` header or generated UUID)
- Client IP, Method, Path
- Execution Time
- Response Status

Example Log

```

2025-09-08 05:22:10 INFO todoapp [8a9b3c] -> 127.0.0.1 GET /todos
2025-09-08 05:22:10 INFO todoapp [8a9b3c] <- GET /todos 200 3.42ms

```

Middleware Features

- Adds response headers:
 - `X-Request-ID` → Trace request across logs
 - `X-Process-Time-ms` → Request duration
- Catches exceptions and logs errors with stack trace.

Example Middleware Code

```

from utils.logging_config import setup_logging

logger = setup_logging()
startup_time = datetime.utcnow()

```

```
# Logging Middleware

@app.on_event("startup")
def on_startup():
    global startup_time
    startup_time = datetime.utcnow()
    logger.info("Application startup")

@app.middleware("http")
async def log_requests(request: Request, call_next):
    request_id = request.headers.get("X-Request-ID", str(uuid.uuid4()))
    start = time.perf_counter()
    client = request.client.host if request.client else "unknown"
    method = request.method
    path = request.url.path

    logger.info(f"[{request_id}] -> {client} {method} {path}")
    try:
        response = await call_next(request)
    except Exception as ex:
        duration_ms = (time.perf_counter() - start) * 1000
        logger.exception(f"[{request_id}] !! {method} {path} failed in
{duration_ms:.2f}ms: {ex}")
        raise
    duration_ms = (time.perf_counter() - start) * 1000
    logger.info(f"[{request_id}] <- {method} {path} {response.status_code}
{duration_ms:.2f}ms")
    response.headers["X-Request-ID"] = request_id
    response.headers["X-Process-Time-ms"] = f"{duration_ms:.2f}"
    return response
```

Health Checks ([routers/healthRouter.py](#))

Health checks are essential for monitoring and orchestration tools like Kubernetes, Docker Swarm, or AWS ALB.

Endpoint

```
GET /health
```

Example Response

✓ When app is healthy:

```
{
  "status": "ok"
}
```

✗ When app is simulated as down:

```
{
  "status": "down"
}
```

The `_is_healthy` flag can be toggled during development/testing to simulate downtime.

Example Code

```
from fastapi import APIRouter
from starlette.responses import JSONResponse

router = APIRouter()

# dev-only toggle to simulate downtime
_is_healthy = True

@router.get("/")
def health():
    if _is_healthy:
        return JSONResponse({"status": "ok"}, status_code=200)
    return JSONResponse({"status": "down"}, status_code=503)
```

TodoApp with MongoDB (Async DB)

We replaced JSON file storage with MongoDB using the Motor driver (async Mongo client).

Database Connection (`db/mongo.py`)

```
from motor.motor_asyncio import AsyncIOMotorClient
import os

MONGO_URL = os.getenv("MONGO_URL", "mongodb://localhost:27017/fastapi_db")
client = AsyncIOMotorClient(MONGO_URL)
database = client.fastapi_db
```

```
user_collection = database.get_collection("users")
todo_collection = database.get_collection("todos")
```

Explanation:

- `AsyncIOMotorClient` → async driver for MongoDB.
- `MONGO_URL` → connection string (default: `mongodb://localhost:27017/fastapi_db`).
- `user_collection` and `todo_collection` → references to MongoDB collections.

This makes DB queries non-blocking, perfect for FastAPI's async model.

Todo Service (`services/todoService.py`)

Now, all CRUD operations run against MongoDB.

Get All Todos

```
async def get_all_todos():
    todos = []
    async for todo in todo_collection.find():
        todo["id"] = str(todo["_id"]) # convert ObjectId → string
        todos.append(todo)
    return todos
```

- Uses `async for` to iterate over MongoDB cursor.
 - `_id` is Mongo's primary key, but we also keep a custom incrementing integer `id` for compatibility.
-

Create Todo

```
async def create_todo(data):
    last = await todo_collection.find().sort("id", -1).limit(1).to_list(length=1)
    new_id = (last[0]["id"] if last else 0) + 1
    todo = {"id": new_id, "title": data.title, "completed": False}
    result = await todo_collection.insert_one(todo)
    todo["id"] = str(result.inserted_id)
    return todo
```

- To stay compatible with old code, we still maintain an auto-incrementing `id` field.
 - Mongo's `_id` is used as unique identifier, but converted to a string for JSON responses.
-

Update Todo

```

async def update_todo(id: int, data):
    update_data = {}
    if data.title is not None:
        update_data["title"] = data.title
    if data.completed is not None:
        update_data["completed"] = data.completed
    result = await todo_collection.update_one({"id": id}, {"$set": update_data})
    if result.modified_count == 1:
        todo = await todo_collection.find_one({"id": id})
        todo["id"] = str(todo["_id"])
        return todo
    return None

```

- Uses `$set` operator to update only provided fields.
 - Returns updated document if successful.
-

Delete Todo

```

async def delete_todo(id: int):
    await todo_collection.delete_one({"id": id})

```

- Removes the todo by its `id`.
 - Returns nothing (handled at router level).
-

Authentication with MongoDB (`services/authService.py`)

Instead of JSON, users are stored in MongoDB.

Read Users

```

async def read_users():
    users = []
    async for user in user_collection.find():
        user["id"] = str(user["_id"])
        users.append(user)
    return users

```

- Iterates through MongoDB `users` collection.
-

Get User

```

async def get_user(username: str):
    user = await user_collection.find_one({"username": username})
    if user:
        user["id"] = str(user["_id"])
        return user
    return None

```

- Finds user by username (used for login).

Signup User

```

async def signup_user(username: str, password: str):
    user = await get_user(username)
    if user:
        raise HTTPException(status_code=400, detail="Username already exists")
    last = await user_collection.find().sort("id", -1).limit(1).to_list(length=1)
    new_id = (last[0]["id"] if last else 0) + 1
    user_data = {
        "id": new_id,
        "username": username,
        "hashed_password": pwd_context.hash(password),
        "role": "user"
    }
    result = await user_collection.insert_one(user_data)
    user_data["id"] = str(result.inserted_id)
    return user_data

```

- Passwords are stored securely using `bcrypt`.
- Users default to "user" role unless created as "admin".

Main Application (`main.py`)

```

from passlib.context import CryptContext
from db.mongo import user_collection

pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

async def init_admin_user():
    admin = await user_collection.find_one({"username": "admin"})
    if not admin:
        admin_user = {
            "id": 1,
            "username": "admin",
            "hashed_password": pwd_context.hash("admin"),
            "role": "admin"
        }

```

```
        await user_collection.insert_one(admin_user)

@app.on_event("startup")
async def startup_event():
    await init_admin_user()
```

- Ensures that an admin user is auto-created if DB is empty.
- Default credentials: `admin / admin`.

Requirements (`requirements.txt`)

```
fastapi
uvicorn
python-jose
passlib
bcrypt==3.2.0
motor          # async MongoDB driver
python-dotenv  # load .env
python-multipart # needed for form data (OAuth2 login)
```

Benefits of MongoDB Integration

- Scalable: JSON file storage replaced with a real database.
- Asynchronous: Non-blocking queries improve performance.
- Authentication: Secure password storage & JWT-based access.
- Admin Bootstrapping: Ensures system always has an admin user.

FastAPI Todo App - Dockerized

Overview

This project is a FastAPI-based Todo application featuring JWT authentication and role-based authorization, containerized with Docker and MongoDB.

1. Building and Running FastAPI with MongoDB Using Docker (manually)

This is the "low-level" way before we use Docker Compose.

Step 1: Build Docker image

Inside the project root where your `Dockerfile` is located, run:

```
docker build -t fastapi-todo .
```

This creates an image named `fastapi-todo` from your `Dockerfile`.

Step 2: Create a Docker network

We need a common network so FastAPI and MongoDB can talk to each other:

```
docker network create todo_network
```

Step 3: Run MongoDB container

```
docker run -d --name mongodb --network todo_network -p 27017:27017 -v  
mongo_data:/data/db mongo:latest
```

This starts MongoDB:

- Runs detached (`-d`)
 - Joins `todo_network`
 - Persists data in a Docker volume `mongo_data`
-

Step 4: Run FastAPI container

```
docker run -d --name fastapi_todo --network todo_network -p 8000:8000 -e  
MONGO_URL=mongodb://mongodb:27017/fastapi_db -e SECRET_KEY=mysecretkey -e  
ALGORITHM=HS256 -e ACCESS_TOKEN_EXPIRE_MINUTES=30 fastapi-todo
```

This starts your FastAPI app:

- Runs detached (`-d`)
- Joins the same network (`todo_network`)
- Talks to MongoDB using the container name `mongodb` (internal DNS resolution in Docker network)

Now you can open:

- `http://localhost:8000/docs` to test the API.
-

2. Using Docker Compose (simpler way)

Instead of running separate commands, we use `docker-compose.yml` (the one you already wrote).

Step 1: Build and start services

```
docker-compose up --build -d
```

This will:

- Build your `fastapi-todo` image
- Start both `fastapi` and `mongo` containers
- Automatically create a network for them
- Expose ports (`8000` for FastAPI, `27017` for Mongo)

Step 2: Check running containers

```
docker ps
```

You should see `fastapi_todo` and `mongodb`.

Step 3: Stop services

```
docker-compose down
```

This stops and removes the containers but **keeps the volume (`mongo_data`)**, so your MongoDB data persists.

If you also want to remove volumes:

```
docker-compose down -v
```

Key Difference

- **Manual way:** You create network, run MongoDB, run FastAPI, pass env vars manually.
- **Docker Compose way:** Single command to bring up/down services. Automatically handles networks and environment variables.

Debugging a FastAPI Application with VS Code

Step-by-Step Instructions

1. Open Project in VS Code

- Launch VS Code.

- Open your FastAPI project folder.

2. Setup Debug Configuration in VS Code

- Go to the **Run and Debug** panel.
- Click "**Add Configuration...**" or "**create a launch.json file**".
- Choose "**Python Debugger**" and select FastAPI.
- Provide the path to your main application file.

Example `launch.json` configuration:

```
{
  // Use IntelliSense to learn about possible attributes.
  // Hover to view descriptions of existing attributes.
  // For more information, visit: https://go.microsoft.com/fwlink/?
linkid=830387
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Python Debugger: FastAPI",
      "type": "debugpy",
      "request": "launch",
      "module": "uvicorn",
      "args": [
        "main:app",
        "--reload"
      ],
      "jinja": true
    }
  ]
}
```

3. Attach Debugger

- Select "**Python Debugger: FastAPI**" from the configuration dropdown.
- Click the green **Run** button () to start debugging.

4. Set Breakpoints in Code

- Open your Python source files.
- Click in the margin to set breakpoints where you want to pause execution.

5. Trigger Breakpoints by Sending Requests to the API

- Use your browser or tools like `curl` or Postman to send requests to your API endpoints.
- Example:

```
curl http://localhost:8000
```

- When a breakpoint is hit, VS Code will pause execution so you can inspect variables and call stacks.
-